

3. Develop a C program using `fork()` that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as `ps`, `top`, and `/proc`. Document the observations.

```
venkatesh@LAPTOP-LL9RQ023:~$ cat week3.c
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    printf("Before Fork()
");
    printf("Current Process PID: %d
", getpid());

    pid = fork();

    if (pid < 0) {
        printf("Fork failed
");
        return 1;
    }
    else if (pid == 0) {
        printf("Child PID: %d
", getpid());
        printf("Parent PID: %d
", getppid());
        printf("Child is Running...
");

        sleep(3);

        printf("Child Process Terminating...
");
        exit(0);
    }
    else {
        printf("Parent PID: %d
", getpid());
        printf("Child PID: %d
", pid);
```

```
        printf("Parent is waiting for Child...\n");

        wait(NULL);

        printf("Child Finished.\n");

        printf("Parent Process Resuming...\n");
    }

    return 0;
}
```

venkatesh@LAPTOP-LL9RQ023:~\$ gcc week3.c

venkatesh@LAPTOP-LL9RQ023:~\$./a.out

Before Fork()

Current Process PID: 645

Parent PID: 645

Child PID: 646

Parent is waiting for Child...

Child PID: 646

Parent PID: 645

Child is Running...

Child Process Terminating...

Child Finished.

Parent Process Resuming...

venkatesh@LAPTOP-LL9RQ023:~\$